CAN201 Introduction to Networking

Coursework: Lightweight Data Transfer Based on STEP Protocol

Team Members

Name (Pinyin)	Student ID
Yaokai Yang	2359830
Xiang Lin	2362086
Xinyang Zhang	2360067
Yunxuan Wu	2361506
Haoran Zheng	2359982

Xi'an Jiaotong-Liverpool University
School of Advanced Technology

November 17, 2025

Lightweight Data Transfer: A Program Based on Simple Transfer and Exchange Protocol

Yaokai Yang
School of Advanced Technology
Xi'an Jiaotong-Liverpool University
SuZhou, China
yaokai.yang23@student.xjtlu.edu.cn

Xiang Lin
School of Advanced Technology
Xi'an Jiaotong-Liverpool University
SuZhou, China
xiang.Lin23@student.xjtlu.edu.cn

Xinyang Zhang
School of Advanced Technology
Xi'an Jiaotong-Liverpool University
SuZhou, China
xinyang.zhang23@student.xjtlu.edu.cn

Yunxuan Wu
School of Advanced Technology
Xi'an Jiaotong-Liverpool University
SuZhou, China
yunxuan.wu23@student.xjtlu.edu.cn

Haoran Zheng
School of Advanced Technology
Xi'an Jiaotong-Liverpool University
SuZhou, China
haoran.zheng23@student.xjtlu.edu.cn

***Abstract*—This project presents a lightweight Python client for the Simple Transfer and Exchange Protocol (STEP), accompanied by a fully corrected version of the defective server framework provided in the lab. The client implements token-based authentication, upload-plan negotiation, chunked file transfer, and MD5 verification, while optimizing file reading through a generator-based streaming mechanism to minimize memory usage. Timeout-based retransmission is incorporated to enhance reliability in unstable or resource-limited environments. A modular design separates networking, authentication, and file-processing logic, improving clarity and maintainability. Experimental evaluation across multiple edge cases confirms the correctness and robustness of the implementation. The resulting system offers a practical and reusable reference for students learning application-layer protocol design and reliable file-transfer mechanisms. We have open-sourced our code on GitHub. The address of the code repository is https://github.com/WeneShan/TCPclient. You can find the complete reproduction steps in this repository.**

***Index Terms*—STEP, File Transfer, Authentication, TCP, MD5**

I. Introduction

With the rapid development of the Internet, cross-device file sharing has become an essential requirement in both everyday use and professional workflows. This project focuses on the Simple Transfer and Exchange Protocol (STEP) and aims to implement a lightweight Python client based on socket programming to support file upload and download under this protocol.

The major tasks of the project are as follows:

1) Fix all syntax and logic errors in the given server-side code.
2) Implement the authorization mechanism, including token retrieval and token-based request validation.
3) Obtain the file upload plan, upload the file in blocks, and verify its integrity using MD5.

These tasks involve three main challenges: analysing and implementing an unfamiliar protocol, designing a simple but effective token-based authentication workflow, and building a reliable chunked-upload mechanism that handles timeouts and retransmissions. The goal is to achieve a stable and efficient file-transfer system using only standard Python libraries.

Although developed in a laboratory setting, the resulting client can be directly applied to lightweight scenarios such as IoT gateways for firmware updates, portable medical devices uploading examination data, or edge-computing cameras synchronizing local media. These applications all require small memory usage and high reliability, which match the design philosophy of this project.

The contribution of this work is the delivery of a clean and fully functional STEP client library that integrates authentication, token management, chunked transfer, and file integrity verification. The debugging process and implementation details are documented clearly so that future learners can reuse, extend, or study this client as a reference for designing application-layer protocols.

II. Related Work

Modern computer networking builds on a layered architecture in which each layer provides distinct yet cooperative functions. At the application layer, protocols such as HTTP define how clients and servers exchange information using standardized methods like GET, POST, and DELETE. Its stateless design simplifies interaction and improves scalability, forming the foundation of RESTful APIs widely used today [1]. Likewise, the Domain Name System (DNS) plays a vital role by translating human-readable domain names into IP addresses through a distributed hierarchy, ensuring both efficiency and global reliability [1].

At the transport layer, TCP and UDP are the two dominant protocols supporting end-to-end data delivery. UDP offers fast but unreliable transmission, whereas TCP guarantees reliable, ordered delivery through acknowledgements and congestion

control. Mechanisms such as Additive Increase and Multiplicative Decrease (AIMD), slow start, and explicit congestion notification (ECN) enable TCP to balance throughput and stability under varying network conditions [3].

Building on these foundations, the Simple Transfer and Exchange Protocol (STEP) is introduced as a lightweight, TCP-based application protocol for structured and reliable file transfer [4]. Using JSON as its message format, STEP defines operations such as LOGIN, SAVE, GET, UPLOAD, and DOWNLOAD. Token-based authentication and MD5 verification ensure that transmitted files remain both secure and verifiable [4].

To realize these mechanisms in practice, socket programming acts as the bridge between the application and transport layers. As demonstrated in the CAN201 labs, sockets allow programs to create endpoints, bind to addresses and ports, and communicate over TCP or UDP [5]. Within the STEP framework, JSON-formatted messages and binary payloads are encoded as byte streams and exchanged through TCP sockets, following the same client–server communication model used in broader network applications. The CAN201 labs also illustrate how file handling and the `struct` module support binary serialization for network transmission [6], [7].

Overall, the STEP client demonstrates how key networking principles, such as TCP reliability [8], structured data exchange [9], and socket-based communication, combine to support an effective and practical file transfer system.

III. Design

A. C/S Network Architecture & Code Workflow

The networking project requires a complete client-server (C/S) file transfer system based on the given STEP protocol and server framework. Our group therefore designed the overall architecture from an end-to-end perspective: the server side must be debugged and stabilized, while the client side must reliably implement authentication, upload planning, block-based file transfer, and integrity verification. Figure 1 summarizes the resulting communication logic between the client and server, and the main design decisions are reflected in the workflow described below.

1) **Connection Initialization:** The client initiates the communication by creating a TCP socket and invoking `connect()`, while the server passively waits for incoming connections using its listening socket.
2) **User Authentication:** Once the TCP channel is ready, the client sends a `LOGIN` request containing its student ID and the MD5 hash of it. The server verifies the credentials and returns a session token, which must be included in all subsequent protocol messages.
3) **Upload Plan Negotiation:** After authentication, the client issues a `SAVE` request to initiate a new upload session. The server responds with the file identifier, block size, and total block count, allowing both sides to agree on a consistent segmentation plan before transmission begins.
4) **Block File Uploading:** With the upload plan established, the client divides the file into multiple blocks and sends them using repeated `UPLOAD` requests, each carrying the block index and its corresponding binary payload. The server stores the received blocks and records their completion status, enabling reliable reconstruction even under parallel or out-of-order delivery.
5) **MD5 Verification and Termination:** After all blocks are received, the server reconstructs the file and computes its MD5 checksum before returning the value to the client. The client compares this checksum with its local result, and if they match, it issues a `BYE` request to terminate the session and close the TCP connection cleanly.

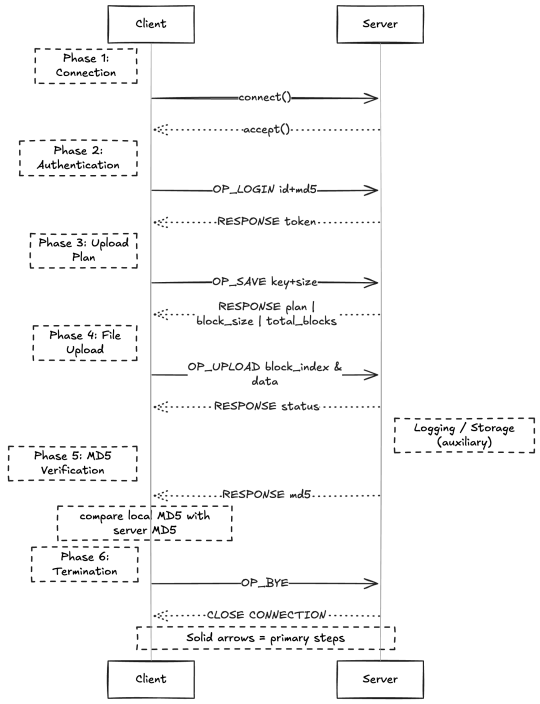


Fig. 1. Client–Server Communication Logic Chain. The process consists of six phases: (1) Connection establishment between client and server, (2) Authentication via LOGIN and token issuance, (3) Upload plan negotiation including block size and total block count, (4) Block-based file uploading with corresponding status responses, (5) Server-side MD5 computation and client-side verification, and (6) Session termination using the BYE operation and connection closure. Solid arrows denote primary protocol exchanges, while dashed arrows represent auxiliary operations such as logging or internal state updates.

B. Pseudocode

This section presents the pseudocode of the client program to explain how the main modules work together in the upload workflow. It consists of two parts. One is the main workflow algorithm containing the whole lifetime procedure of the client, which is connection establishment, authentication and login, upload plan negotiation, file uploading and session termination, as shown in Algorithm 1. Another is the block-upload algorithm that contains the essential protocol behaviour: block transmission, retransmission handling, and MD5 validation, as illustrated in Algorithm 2.

Algorithm 1 Main Client Workflow

```
function MAIN(server_ip, server_port, file_path, custom_key)
    client ← new STEPFileClient(server_ip, server_port)
    if not client.connect() then
        return
    end if{authorization}
    if not client.login(student_id) then
        client.close()
        return
    end if{upload plan negotiation}
    (file_key, block_size, total_blocks, token)
    ← client.request_plan(file_path, custom_key) {block upload and verification}
    ok ← UPLOAD_BLOCKS(client.socket, file_path, file_key, block_size, total_blocks, token)
    if ok then
        client.send_bye()
    end if
    client.close()
end function=0
```

Algorithm 2 Block Upload with Retransmission and MD5 Check

```
function UPLOAD_BLOCKS(socket, file_path, file_key, block_size, total_blocks, token)
    for block_idx ← 0 to total_blocks −1 do
        data ← READ_BLOCK(file_path, block_idx, block_size)
        attempts ← 0
        success ← False
        while not success and attempts < MAX_RETRY do
            payload ← {key: file_key, block_index: block_idx}
            SEND(socket, OP_UPLOAD, payload, data, token)
            response ← RECV(socket, timeout)
            if response.status = 200 then
                success ← True
            else if response.status = 406 or response = timeout then
                attempts ← attempts +1 {retransmit current block}
            else
                return False {unexpected error}
            end if
        end while
        if not success then
            return False
        end if
    end for{MD5 integrity verification}
    remote_md5 ← response.md5
    local_md5 ← CALC_LOCAL_MD5(file_path)
    return (local_md5 = remote_md5)
end function=0
```

Together, these two algorithms cover all functional stages of the upload workflow while maintaining a concise and protocol-oriented description.

IV. Implementation

A. Development Environment and Tools

The client was developed and tested using Python 3 on a MacBook laptop equipped with an Apple M4 processor and 24 GB of RAM. Visual Studio Code, together with official Python tooling, served as the primary development environment. Only standard Python libraries, including `socket`, `json`, `struct`, `hashlib`, `os`, and `argparse`, were used so that the program remains fully compatible with the laboratory execution environment.

These tools allowed the implementation to remain lightweight while still supporting efficient file I/O, binary packing, and TCP communication.

B. Server Debugging

The server skeleton was provided by the lecturer and intentionally contained multiple faults. Debugging the server became an important component of the implementation, ensuring the client could communicate with the backend in a stable and correct protocol.

The most significant issues detected and resolved include:

- Missing `import struct`, causing runtime `NameError`;
- Opening files in text mode instead of binary, corrupting MD5 computation;
- Inconsistent naming (`op_save` vs. `OP_SAVE`);
- Incorrect socket type (`SOCK_DGRAM`) used instead of `SOCK_STREAM`;
- Threads created but never executed (missing `t.start()`);
- Mis-typed identifier (`tcp-listener`) interpreted as subtraction.

Correcting these issues produced a stable and fully functional STEP server compatible with our client.

In addition, during the debugging phase, a small extension to the protocol as an easter egg was discovered in the server codebase. In case that the JSON data contains a "`direction`" field with a value of "`EARTH`", the server responses with a customized error message of "DO NOT ANSWER", displaying the lecturer's sense of humor.

C. Client Implementation Workflow

The client follows a well-defined workflow that mirrors the STEP protocol specification. Rather than implementing a monolithic script, we deliberately separated the program into several chunks, each of which corresponds to a specific protocol functionality.

1) Parameter Parsing: Command-line parameters (server IP, port, file, and optional custom key) are parsed via `argparse`. Default values (`127.0.0.1:1379`) allow immediate local testing without additional configuration.

2) TCP Connection Establishment: An IPv4 stream socket (`AF_INET`, `SOCK_STREAM`) is created. Connection failures are caught early and reported to the user. Once connected, the program initializes its authentication and file-transfer modules.

3) User Authentication: Authentication follows a secure token-based workflow:

- Student ID is hashed via MD5;
- A `LOGIN` request is sent with the username and hashed password;
- The server returns a session token that is used for all subsequent operations.

Invalid credentials or 4xx responses immediately terminate the workflow. A small easter-egg command is also included for the special username "`YeWenjie`", demonstrating extensibility of the framework.

4) Upload Plan Retrieval: The client issues a `SAVE` request and receives the upload plan: file key, file size, block size, and total block count. This negotiation step guarantees that both client and server share an identical understanding of how the file is segmented, thereby preventing out-of-sync transfer states.

5) Efficient File Block Reading: A key design choice of our implementation is the use of a generator-based block reader. Instead of loading the entire file into memory, the client reads one block at a time using standard file I/O. The generator produces one block per iteration and automatically handles the final partial block.

This lazy evaluation design significantly reduces memory footprint and avoids unnecessary system calls. It is particularly advantageous for large file uploads, providing near-constant memory usage.

6) Block Upload with Retransmission Logic: Blocks are uploaded sequentially using the `UPLOAD` operation. A metadata header containing the file key and block index is attached. With a 20-second timeout, the block is retransmitted once a timeout occurs or `406` response is received. A successful response (`200`) advances the cursor to the next block.

This design implements reliability at the application layer and mirrors well-known retransmission concepts in TCP (e.g. timeout-based recovery), adapted to STEP protocol's requirements.

During this process, a dynamic progress bar displays real-time speed in MB/s, elapsed time, and percentage completion.

7) MD5 Integrity Verification: Once all blocks are received, the server reconstructs the file and returns its MD5 checksum. The client calculates its own MD5 and compares the two values. A match indicates complete and correct transfer; otherwise, corruption is reported.

8) Session Termination and Cleanup: The client sends a `BYE` request to formally close the STEP session and then shuts down the TCP socket safely, ensuring proper release of resources even in the presence of exceptions.

D. Difficulties and Solutions

When designing and implementing the project, we met several difficulties and challenges. They include the difficulty ensuring efficient file reading without high memory consumption, implementing robust retransmission logic that correctly handles timeouts, debugging a partially-broken server reference implementation, and designing a modular architecture that reflects the protocol design.

Each difficulty was addressed with targeted strategies such as generator-based file processing, timeout-based retry loops, and systematic debugging using stack traces and incremental testing.

V. Testing and Results

A. Testing environment

Given that physical networks may contain numerous uncontrollable factors that could potentially interfere with the reproducibility of experiments, we opted to conduct testing on both the client and server sides within two virtual machines.

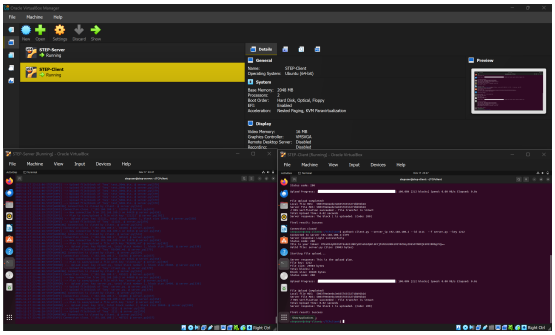


Fig. 2. Test Environment Screenshot. We ran two Ubuntu 22.04.5 guests on a Windows 11 host in VirtualBox. For network configuration, both servers were placed on the same host-only network. The server VM's IP was set to `192.168.100.2/24`, while the client virtual machine was assigned the IP address `192.168.100.3/24`.

B. Testing steps

We evaluated two primary aspects of the implementation of this protocol. The first part concerns the correctness of the execution of the protocol under various conditions. The second part involves testing factors that impact protocol performance. We employed a method of conducting multiple tests and taking the average to eliminate random errors and obtain representative, typical results.

1) Successful Execution of Server: After the successful debugging and fix of all errors in previous steps, the server was ran to confirm its error-free state. It should display a "Server is ready" message to broadcast its healthy state.

2) Functionality and Correctness: We designed several Python scripts, running within a virtual machine on the client side, to evaluate the functionality and correctness of the protocol, covering several corner cases. Labeled A1 to A4, the purpose of the tests are listed as follows:

1) A1: Single-client standard file upload (basic functionality)
2) A2: 0-byte / 1-byte file upload
3) A3: File size not a multiple of block_size (tail block handling)
4) A4: Duplicate Uploads (Same File Name/Same Content) / Overwrite Policy

We ran the tests and collected the results in JSON files.

3) Protocol Performance: We also conducted a benchmark with variable file sizes. The folder labeled C within the test directory contains Python script (running within a virtual machine on the client side) designed to evaluate protocol performance. This test aims to clarify the relationship between uploaded file size and protocol performance. As file size increases, the server's upload scheduling—such as block_size—also adjusts accordingly. Continuous reading and transmission of large files may exhibit different behaviors in channel utilization, thereby impacting the protocol's throughput and transfer speed.

C. Testing results

1) Successful Execution of Server: The server script ran correctly on port 1379 without errors. Upon execution, the text "Server is ready" was correctly displayed on the terminal emulator demonstrated by Figure 3.

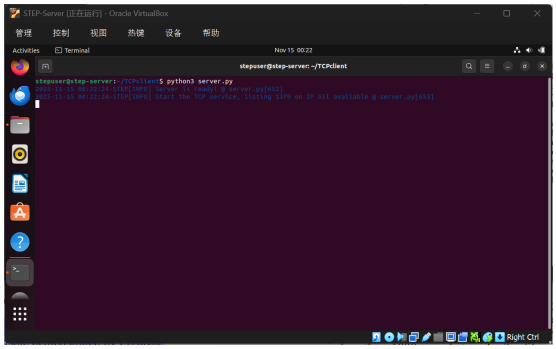
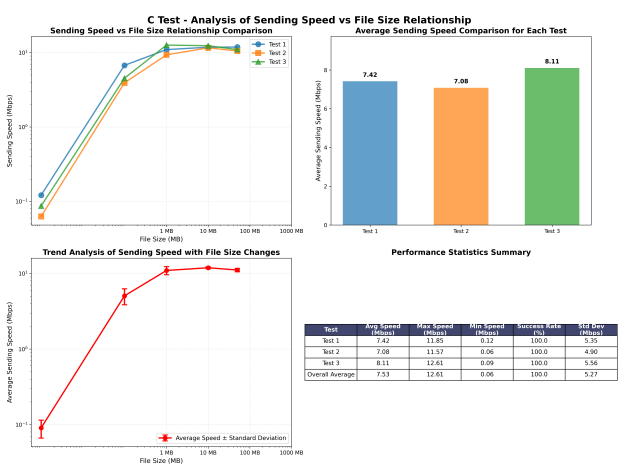


Fig. 3. Screenshot of execution of server.py, indicating success.

Test code	Result	Comment
A1	pass	The client performed normally in the standard file upload test.
A2	pass	The client rejected the upload of the 0b file. The 1kb file was uploaded normally.
A3	pass	The client performed normally in the test.
A4	pass	The client rejected the upload of the duplicated files (with same key).

TABLE I
FUNCTIONALITY AND CORRECTNESS TESTS RESULTS AND COMMENTS

2) Functionality and Correctness: Table I demonstrates that our STEP implementation functioned correctly even under certain corner cases or adverse conditions, indicating that this implementation possesses a degree of robustness.



Test	Avg Speed (Mbps)	Max Speed (Mbps)	Min Speed (Mbps)	Success Rate (%)	Std Dev (Mbps)
Test 1	7.42	11.85	0.12	100.0	5.35
Test 2	7.08	11.57	0.06	100.0	4.90
Test 3	8.11	12.61	0.09	100.0	5.56
Overall Average	7.53	12.61	0.06	100.0	5.27

Fig. 4. Analysis of Sending Speed vs File Size Relationship. We conducted three experiments and plotted the images using the matplotlab library. We have adopted a logarithmic scale to represent file sizes in order to better illustrate the trend of change.

3) Protocol Performance: As shown in Figure 4, our benchmark shows the transmission speed exhibits a very interesting trend as the file size increases.

We suspect that with small files, the fixed and lengthy RTT value leads to insufficient utilization of the TCP channel. As file sizes increase, parallel TCP pipelines enhance channel utilization, masking the communication delay overhead caused by RTT. Once TCP channel utilization reaches a certain threshold, the overall transmission rate ceases to increase. This hypothesis requires further experimentation to validate.

VI. CONCLUSION

In this study, we fully developed, optimized, and implemented a file transfer client based on the STEP protocol. By debugging the defective server-side code, we stabilized the service and successfully achieved functionalities including login authentication, token acquisition, chunked upload, and download with MD5 verification. The modular design encapsulates authentication, streaming I/O, and integrity verification as independent components, enabling the client to be directly reused in resource-constrained scenarios such as IoT gateways and portable medical devices.

In future work, the file-reading algorithm could be further refined to dynamically adapt to diverse hardware configurations and thereby push upload performance to its peak.

ACKNOWLEDGMENT

Yaokai Yang (2359830) contributes 20% to the project, Xiang Lin (2362086) contributes 20% to the project, Xinyang Zhang (2360067) contributes 20% to the project, Yunxuan Wu (2361506) contributes 20% to the project and Haoran Zheng (2359982) contributes 20% to the project.

REFERENCES

[1] W. Fan and F. Cheng, "CAN201 Lecture 3: Introduction to Networking [PowerPoint slides]," Xi'an Jiaotong–Liverpool University, 2025.
[2] F. Cheng and G. Boateng, "CAN201 Lecture 5: Introduction to Networking [PowerPoint slides]," Xi'an Jiaotong–Liverpool University, 2025.
[3] F. Cheng and G. Boateng, "CAN201 Lecture 6: Introduction to Networking [PowerPoint slides]," Xi'an Jiaotong–Liverpool University, 2025.
[4] CAN201 Teaching Team, "Simple Transfer and Exchange Protocol (STEP) v1.2 [Course handout]," Xi'an Jiaotong–Liverpool University, 2025.
[5] F. Cheng and G. Boateng, "CAN201 Lab 4: Socket Programming [Lab slides]," Xi'an Jiaotong–Liverpool University, 2025.
[6] F. Cheng and G. Boateng, "CAN201 Lab 5: UDP/TCP Programming, Wireshark, and File Operations [Lab slides]," Xi'an Jiaotong–Liverpool University, 2025.
[7] F. Cheng and G. Boateng, "CAN201 Lab 6: Parallel computing and Data Structure [Lab slides]," Xi'an Jiaotong–Liverpool University, 2025.
[8] J. Postel, "Transmission Control Protocol," *RFC 793*, Internet Engineering Task Force, Sept. 1981. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc793
[9] L. Richardson and S. Ruby, *RESTful Web Services*. O'Reilly Media, 2007. [Online]. Available: https://www.oreilly.com/library/view/restful-web-services/9780596529260/